iroh 개발자 가이드 및 실용 예제

개요

이 문서는 iroh를 활용하여 P2P 애플리케이션을 개발하는 개발자들을 위한 포괄적인 가이드입니다. 기본적인 연결 설정부터 고급 활용법, 그리고 실제 사용 사례까지 단계별로 설명합니다.

목차

1. iroh 활용 기본 가이드
2. 고급 활용 및 확장 가이드
3. 실제 사용 사례 및 예제 코드

1. iroh 활용 기본 가이드

1.1 기본 연결 설정 및 사용법

프로젝트 설정

먼저 `Cargo.toml` 에 iroh 의존성을 추가합니다:

```
[dependencies]

iroh = "0.28"

tokio = { version = "1.0", features = ["full"] }

anyhow = "1.0"

tracing = "0.1"
```

```
tracing-subscriber = "0.3"
```

기본 Endpoint 생성

```rust
use iroh::{Endpoint, NodeAddr};
use anyhow::Result;

#[tokio::main]
async fn main() -> Result<()> {
// 로깅 초기화
tracing_subscriber::fmt::init();
// 기본 설정으로 Endpoint 생성
let endpoint = Endpoint::builder()
.bind()
.await?;
println!("노드 ID: {}", endpoint.node_id());
println!("바인딩된 주소: {:?}", endpoint.bound_sockets());
// 직접 주소 확인
let direct_addrs = endpoint.direct_addresses().next().await;
if let Some(Some(addrs)) = direct_addrs {
println!("직접 주소들: {:?}", addrs);
}
// 홈 릴레이 서버 확인
```

```
let home_relay = endpoint.home_relay().initialized().await;
println!("홈 릴레이: {}", home_relay);
Ok(())
}
```

연결 설정 예제

서버 측 (연결 수락):

```
use iroh::{Endpoint, endpoint::Connecting};
use anyhow::Result;

const ALPN: &[u8] = b"my-app/1.0";

async fn run_server() -> Result<()> {
let endpoint = Endpoint::builder()
.alpns(vec![ALPN.to_vec()])
.bind()
.await?;
println!("서버 시작됨. 노드 ID: {}", endpoint.node_id());
// 들어오는 연결 처리
while let Some(connecting) = endpoint.accept().await {
tokio::spawn(handle_connection(connecting));
}
Ok(())
```

```
}

async fn handle_connection(connecting: Connecting) -> Result<()> {
let connection = connecting.await?;
let remote_node_id = connection.remote_node_id()?;
println!("새 연결: {}", remote_node_id);
// 양방향 스트림 수락
let (mut send, mut recv) = connection.accept_bi().await?;
// 간단한 에코 서버
let mut buffer = [0u8; 1024];
loop {
match recv.read(&mut buffer).await? {
Some(n) if n > 0 => {
println!("받은 데이터: {} 바이트", n);
send.write_all(&buffer[..n]).await?;
}
_ => break,
}
}
println!("연결 종료: {}", remote_node_id);
Ok(())
}
```

클라이언트 측 (연결 시작):

```rust
use iroh::{Endpoint, NodeAddr};
use anyhow::Result;

async fn run_client(server_node_addr: NodeAddr) -> Result<()> {
let endpoint = Endpoint::builder()
.bind()
.await?;
println!("클라이언트 시작됨. 노드 ID: {}", endpoint.node_id());
// 서버에 연결
let connection = endpoint.connect(server_node_addr, ALPN).await?;
println!("서버에 연결됨");
// 양방향 스트림 열기
let (mut send, mut recv) = connection.open_bi().await?;
// 메시지 전송
let message = b"Hello, iroh!";
send.write_all(message).await?;
send.finish()?;
// 응답 받기
let response = recv.read_to_end(1024).await?;
println!("서버 응답: {}", String::from_utf8_lossy(&response));
Ok(())
```

```
}
```

1.2 일반적인 사용 패턴 및 베스트 프랙티스

1. 연결 풀 관리

여러 노드와의 연결을 효율적으로 관리하기 위한 연결 풀 패턴:

```
use std::collections::HashMap;

use std::sync::Arc;

use tokio::sync::RwLock;

use iroh::{Endpoint, NodeAddr, endpoint::Connection};


pub struct ConnectionPool {

endpoint: Endpoint,

connections: Arc<RwLock<HashMap<iroh_base::NodeId, Connection>>>,

alpn: Vec<u8>,

}


impl ConnectionPool {

pub fn new(endpoint: Endpoint, alpn: &[u8]) -> Self {

Self {

endpoint,

connections: Arc::new(RwLock::new(HashMap::new())),

alpn: alpn.to_vec(),
```

```
}
}
/// 연결 가져오기 (없으면 새로 생성)
pub  async  fn  get_connection(&self, node_addr: NodeAddr) -> Result<Connection>
let  node_id = node_addr.node_id;
// 기존 연결 확인
{
let  connections = self.connections.read().await;
if  let Some(conn) = connections.get(&node_id) {
if  conn.close_reason().is_none() {
return Ok(conn.clone());
}
}
}
// 새 연결 생성
let  connection = self.endpoint.connect(node_addr, &self.alpn).await?;
// 연결 저장
{
let  mut  connections = self.connections.write().await;
connections.insert(node_id, connection.clone());
}
Ok(connection)
}
```

```
/// 연결 제거
pub async fn remove_connection(&self, node_id: iroh_base::NodeId) {
let mut connections = self.connections.write().await;
if let Some(conn) = connections.remove(&node_id) {
conn.close(0u32.into(), b"Connection removed from pool");
}
}
/// 모든 연결 정리
pub async fn close_all(&self) {
let mut connections = self.connections.write().await;
for (_, conn) in connections.drain() {
conn.close(0u32.into(), b"Pool shutdown");
}
}
}
```

2. 메시지 프로토콜 정의

구조화된 메시지 교환을 위한 프로토콜 정의:

```
use serde::{Serialize, Deserialize};
use anyhow::Result;

#[derive(Debug, Serialize, Deserialize)]
pub enum Message {
```

```
Request { id: u64, data: Vec<u8> },
Response { id: u64, result: Result<Vec<u8>, String> },
Notification { event: String, payload: Vec<u8> },
}

impl Message {
/// 메시지를 바이너리로 직렬화
pub fn to_bytes(&self) -> Result<Vec<u8>> {
Ok(postcard::to_stdvec(self)?)
}
/// 바이너리에서 메시지 역직렬화
pub fn from_bytes(data: &[u8]) -> Result<Self> {
Ok(postcard::from_bytes(data)?)
}
/// 스트림으로 메시지 전송
pub async fn send(&self, send: &mut quinn::SendStream) -> Result<()> {
let data = self.to_bytes()?;
let len = data.len() as u32;
// 길이 헤더 전송
send.write_all(&len.to_le_bytes()).await?;
// 메시지 데이터 전송
send.write_all(&data).await?;
Ok(())
```

```rust
}

/// 스트림에서 메시지 수신

pub  async  fn  recv(recv: &mut quinn::RecvStream) -> Result<Self> {

// 길이 헤더 읽기

let  mut  len_buf = [0u8; 4];

recv.read_exact(&mut  len_buf).await?;

let  len = u32::from_le_bytes(len_buf) as usize;

// 메시지 데이터 읽기

let  mut  data = vec![0u8; len];

recv.read_exact(&mut  data).await?;

Self::from_bytes(&data)

}

}
```

3. 비동기 메시지 핸들러

요청-응답 패턴을 위한 비동기 핸들러:

```rust
use std::collections::HashMap;

use tokio::sync::{mpsc, oneshot};



pub  struct MessageHandler {

pending_requests: Arc<RwLock<HashMap<u64, oneshot::Sender<Message>>>>,

next_request_id: Arc<std::sync::atomic::AtomicU64>,
```

```
}

impl MessageHandler {
pub fn new() -> Self {
Self {
pending_requests: Arc::new(RwLock::new(HashMap::new())),
next_request_id: Arc::new(std::sync::atomic::AtomicU64::new(1)),
}
}
/// 요청 전송 및 응답 대기
pub async fn send_request(
&self,
connection: &Connection,
data: Vec<u8>,
) -> Result<Vec<u8>> {
let request_id = self.next_request_id
.fetch_add(1, std::sync::atomic::Ordering::SeqCst);
let (tx, rx) = oneshot::channel();
// 응답 대기자 등록
{
let mut pending = self.pending_requests.write().await;
pending.insert(request_id, tx);
}
```

```rust
// 요청 전송
let (mut send, _) = connection.open_bi().await?;
let request = Message::Request { id: request_id, data };
request.send(&mut send).await?;
send.finish()?;
// 응답 대기
match rx.await? {
Message::Response { result, .. } => {
match result {
Ok(data) => Ok(data),
Err(err) => Err(anyhow::anyhow!("Request failed: {}", err)),
}
}
_ => Err(anyhow::anyhow!("Unexpected response type")),
}
}
/// 들어오는 메시지 처리
pub async fn handle_incoming_message(&self, message: Message) -> Result<()> {
match message {
Message::Response { id, .. } => {
// 응답 메시지 처리
let mut pending = self.pending_requests.write().await;
if let Some(tx) = pending.remove(&id) {
```

```
let _ = tx.send(message);
}
}
Message::Request { id, data } => {
// 요청 메시지 처리 (별도 핸들러에서)
self.handle_request(id, data).await?;
}
Message::Notification { event, payload } => {
// 알림 메시지 처리
self.handle_notification(event, payload).await?;
}
}
Ok(())
}
async fn handle_request(&self, id: u64, data: Vec<u8>) -> Result<()> {
// 요청 처리 로직 구현
// 실제 애플리케이션에서는 비즈니스 로직 호출
println!("처리 중인 요청 ID: {}, 데이터 크기: {}", id, data.len());
Ok(())
}
async fn handle_notification(&self, event: String, payload: Vec<u8>) -> Result
// 알림 처리 로직 구현
println!("알림 수신: {}, 페이로드 크기: {}", event, payload.len());
```

```
Ok(())

}

}
```

1.3 에러 처리 및 예외 상황 대응 방법

1. 포괄적인 에러 타입 정의

```
use thiserror::Error;


#[derive(Error, Debug)]

pub  enum AppError {

#[error("연결 오류: {0}")]

Connection(#[from] iroh::endpoint::ConnectError),

#[error("바인딩 오류: {0}")]

Bind(#[from] iroh::endpoint::BindError),

#[error("스트림 오류: {0}")]

Stream(#[from] quinn::WriteError),

#[error("직렬화 오류: {0}")]

Serialization(#[from] postcard::Error),

#[error("타임아웃 오류")]

Timeout,

#[error("노드를 찾을 수 없음: {node_id}")]

NodeNotFound { node_id: iroh_base::NodeId },
```

```
#[error("프로토콜 오류: {message}")]
Protocol { message: String },
#[error("리소스 부족: {resource}")]
ResourceExhausted { resource: String },
}

pub type AppResult<T> = Result<T, AppError>;
```

2. 재시도 메커니즘

```
use tokio::time::{sleep, Duration};

pub struct RetryConfig {
pub max_attempts: usize,
pub initial_delay: Duration,
pub max_delay: Duration,
pub backoff_factor: f64,
}

impl Default for RetryConfig {
fn default() -> Self {
Self {
max_attempts: 3,
initial_delay: Duration::from_millis(100),
```

```
max_delay: Duration::from_secs(30),
backoff_factor: 2.0,
}
}
}

pub async fn retry_with_backoff<F, Fut, T, E>(
config: &RetryConfig,
mut operation: F,
) -> Result<T, E>
where
F: FnMut() -> Fut,
Fut: std::future::Future<Output = Result<T, E>>,
E: std::fmt::Debug,
{
let mut delay = config.initial_delay;
for attempt in 1..=config.max_attempts {
match operation().await {
Ok(result) => return Ok(result),
Err(e) if attempt == config.max_attempts => return Err(e),
Err(e) => {
tracing::warn!("시도 {}/{} 실패: {:?}", attempt, config.max_attempts, e);
sleep(delay).await;
```

```
delay = std::cmp::min(
Duration::from_millis((delay.as_millis() as f64 * config.backoff_factor) as u64)
config.max_delay,
);
}
}
}
unreachable!()
}

// 사용 예제
async fn connect_with_retry(
endpoint: &Endpoint,
node_addr: NodeAddr,
alpn: &[u8],
) -> AppResult<Connection> {
let config = RetryConfig::default();
retry_with_backoff(&config, || async {
endpoint.connect(node_addr.clone(), alpn).await
.map_err(AppError::Connection)
}).await
}
```

3. 연결 상태 모니터링

```rust
use tokio::sync::watch;

pub struct ConnectionMonitor {
endpoint: Endpoint,
status_tx: watch::Sender<ConnectionStatus>,
_status_rx: watch::Receiver<ConnectionStatus>,
}

#[derive(Debug, Clone)]
pub enum ConnectionStatus {
Disconnected,
Connecting,
Connected { peer_count: usize },
Error { message: String },
}

impl ConnectionMonitor {
pub fn new(endpoint: Endpoint) -> Self {
let (status_tx, status_rx) = watch::channel(ConnectionStatus::Disconnected);
Self {
endpoint,
```

```
status_tx,
_status_rx: status_rx,
}
}
pub fn status_receiver(&self) -> watch::Receiver<ConnectionStatus> {
self.status_tx.subscribe()
}
pub async fn start_monitoring(&self) {
let mut net_report_watcher = self.endpoint.net_report();
let mut direct_addr_watcher = self.endpoint.direct_addresses();
loop {
tokio::select! {
report = net_report_watcher.next() => {
if let Some(Some(report)) = report {
self.handle_net_report(report).await;
}
}
addrs = direct_addr_watcher.next() => {
if let Some(addrs) = addrs {
self.handle_address_change(addrs).await;
}
}
}
```

```
}

}

async fn handle_net_report(&self, report: iroh::net::Report) {

if !report.udp {

let _ = self.status_tx.send(ConnectionStatus::Error {

message: "UDP 트래픽이 차단됨".to_string(),

});

} else {

// 연결 상태 업데이트 로직

let metrics = self.endpoint.metrics();

let peer_count = metrics.magicsock.num_direct_conns_added.get() as usize;

let _ = self.status_tx.send(ConnectionStatus::Connected { peer_count });

}

}

async fn handle_address_change(&self, addrs: Option<Vec<iroh::net::DirectAddr>

if let Some(addrs) = addrs {

tracing::info!("직접 주소 업데이트: {:?}", addrs);

}

}

}
```

4. 우아한 종료 (Graceful Shutdown)

```
use tokio::signal;
```

```rust
use tokio_util::sync::CancellationToken;


pub struct Application {
endpoint: Endpoint,
connection_pool: ConnectionPool,
shutdown_token: CancellationToken,
}


impl Application {
pub async fn run(&self) -> AppResult<()> {
// 종료 신호 처리
let shutdown_token = self.shutdown_token.clone();
tokio::spawn(async move {
let _ = signal::ctrl_c().await;
tracing::info!("종료 신호 수신됨");
shutdown_token.cancel();
});
// 메인 애플리케이션 루프
loop {
tokio::select! {
_ = self.shutdown_token.cancelled() => {
tracing::info!("애플리케이션 종료 중...");
break;
```

```
}
connecting = self.endpoint.accept() => {
if  let Some(connecting) = connecting {
let  token = self.shutdown_token.clone();
tokio::spawn(async  move {
tokio::select! {
_ = token.cancelled() => {
tracing::debug!("연결 처리 중단됨");
}
result = handle_connection(connecting) => {
if  let Err(e) = result {
tracing::error!("연결 처리 오류: {}", e);
}
}
}
});
}
}
}
}
// 정리 작업
self.shutdown().await?;
Ok(())
```

```
}
async fn shutdown(&self) -> AppResult<()> {
tracing::info!("리소스 정리 중...");
// 연결 풀 정리
self.connection_pool.close_all().await;
// Endpoint 종료
self.endpoint.close(0u32.into(), b"Application shutdown").await?;
tracing::info!("애플리케이션 종료 완료");
Ok(())
}
}
```

5. 로깅 및 디버깅

```
use tracing::{info, warn, error, debug, trace};
use tracing_subscriber::{layer::SubscriberExt, util::SubscriberInitExt};

pub fn init_logging() -> AppResult<()> {
tracing_subscriber::registry()
.with(
tracing_subscriber::EnvFilter::try_from_default_env()
.unwrap_or_else(|_| "info,iroh=debug".into()),
)
```

```
.with(tracing_subscriber::fmt::layer())
.init();
Ok(())
}


// 구조화된 로깅 예제
async fn handle_connection_with_logging(connecting: Connecting) -> AppResult<(
let connection = connecting.await?;
let remote_node_id = connection.remote_node_id()?;
info!(
remote_node_id = %remote_node_id,
"새 연결 수락됨"
);
let start_time = std::time::Instant::now();
match process_connection(&connection).await {
Ok(()) => {
let duration = start_time.elapsed();
info!(
remote_node_id = %remote_node_id,
duration_ms = duration.as_millis(),
"연결 처리 완료"
);
}
```

```
Err(e) => {
error!(
remote_node_id = %remote_node_id,
error = %e,
"연결 처리 실패"
);
return Err(e);
}
}
Ok(())
}


async fn process_connection(connection: &Connection) -> AppResult<()> {
// 연결 처리 로직
debug!("연결 처리 시작");
// 스트림 통계 로깅
let stats = connection.stats();
trace!(
rtt = ?stats.path.rtt,
cwnd = stats.path.cwnd,
"연결 통계"
);
Ok(())
```

```
}
```

이러한 기본 패턴들을 활용하면 안정적이고 확장 가능한 iroh 기반 애플리케이션을 개발할 수 있습니다. 다음 섹션에서는 더 고급 활용법과 확장 기법을 다루겠습니다.

2. 고

급 활용 및 확장 가이드

2.1 커스텀 Discovery 구현 방법

iroh의 Discovery 시스템은 확장 가능한 아키텍처를 제공하여 커스텀 노드 발견 메커니즘을 구현할 수 있습니다.

Discovery 트레이트 구현

```
use iroh::discovery::{Discovery, DiscoveryItem, DiscoveryError};

use iroh_base::{NodeId, NodeAddr};

use futures_lite::Stream;

use std::pin::Pin;


/// 커스텀 Discovery 서비스 예제 - 중앙 집중식 레지스트리

pub  struct CentralizedDiscovery {

registry_url: String,

http_client: reqwest::Client,

node_id: NodeId,

}
```

```
impl CentralizedDiscovery {
pub fn new(registry_url: String, node_id: NodeId) -> Self {
Self {
registry_url,
http_client: reqwest::Client::new(),
node_id,
}
}
}

#[async_trait::async_trait]
impl Discovery for CentralizedDiscovery {
/// 노드 정보를 중앙 레지스트리에 게시
async fn publish(&self, node_addr: &NodeAddr) -> Result<(), DiscoveryError> {
let url = format!("{}/nodes/{}", self.registry_url, node_addr.node_id);
let payload = serde_json::json!({
"node_id": node_addr.node_id.to_string(),
"relay_url": node_addr.relay_url().map(|u| u.to_string()),
"direct_addresses": node_addr.direct_addresses().collect::<Vec<_>>(),
"timestamp": std::time::SystemTime::now()
.duration_since(std::time::UNIX_EPOCH)
.unwrap()
.as_secs()
```

```
});
let response = self.http_client
.put(&url)
.json(&payload)
.send()
.await
.map_err(|e| DiscoveryError::Other(e.into()))?;
if !response.status().is_success() {
return Err(DiscoveryError::Other(
anyhow::anyhow!("Failed to publish: {}", response.status()).into()
));
}
tracing::info!("노드 정보 게시 완료: {}", node_addr.node_id);
Ok(())
}
/// 중앙 레지스트리에서 노드 정보 조회
async fn resolve(&self, node_id: NodeId) -> Option<BoxStream<Result<DiscoveryI
let url = format!("{}/nodes/{}", self.registry_url, node_id);
let stream = async_stream::stream! {
match self.http_client.get(&url).send().await {
Ok(response) if response.status().is_success() => {
match response.json::<serde_json::Value>().await {
Ok(data) => {
```

```
if  let Some(node_addr) = self.parse_node_addr(&data) {
yield Ok(DiscoveryItem::NodeAddr(node_addr));
}
}
Err(e) => yield Err(DiscoveryError::Other(e.into())),
}
}
Ok(response) => {
yield Err(DiscoveryError::Other(
anyhow::anyhow!("HTTP error: {}", response.status()).into()
));
}
Err(e) => yield Err(DiscoveryError::Other(e.into())),
}
};
Some(Box::pin(stream))
}
/// 노드 변경 사항 구독 (폴링 방식)
async  fn  subscribe(&self) -> Option<BoxStream<DiscoveryItem>> {
let  registry_url = self.registry_url.clone();
let  http_client = self.http_client.clone();
let  stream = async_stream::stream! {
let  mut  last_check = std::time::SystemTime::UNIX_EPOCH;
```

```
let mut interval = tokio::time::interval(std::time::Duration::from_secs(30));
loop {
interval.tick().await;
let url = format!("{}/nodes/changes?since={}",
registry_url,
last_check.duration_since(std::time::UNIX_EPOCH).unwrap().as_secs()
);
match http_client.get(&url).send().await {
Ok(response) if response.status().is_success() => {
match response.json::<Vec<serde_json::Value>>().await {
Ok(changes) => {
for change in changes {
if let Some(node_addr) = self.parse_node_addr(&change) {
yield DiscoveryItem::NodeAddr(node_addr);
}
}
last_check = std::time::SystemTime::now();
}
Err(e) => tracing::error!("구독 파싱 오류: {}", e),
}
}
Ok(response) => {
tracing::error!("구독 HTTP 오류: {}", response.status());
```

```
}
Err(e) => tracing::error!("구독 네트워크 오류: {}", e),
}
}
};
Some(Box::pin(stream))
}
}


impl CentralizedDiscovery {
fn parse_node_addr(&self, data: &serde_json::Value) -> Option<NodeAddr> {
let node_id_str = data.get("node_id")?.as_str()?;
let node_id = NodeId::from_str(node_id_str).ok()?;
let mut node_addr = NodeAddr::new(node_id);
// Relay URL 추가
if let Some(relay_url_str) = data.get("relay_url").and_then(|v| v.as_str()) {
if let Ok(relay_url) = relay_url_str.parse() {
node_addr = node_addr.with_relay_url(relay_url);
}
}
// 직접 주소들 추가
if let Some(direct_addrs) = data.get("direct_addresses").and_then(|v| v.as_arra
for addr_value in direct_addrs {
```

```
if let Some(addr_str) = addr_value.as_str() {
if let Ok(socket_addr) = addr_str.parse() {
node_addr = node_addr.with_direct_addresses([socket_addr]);
}
}
}
}
Some(node_addr)
}
}
```

하이브리드 Discovery 시스템

여러 Discovery 서비스를 조합하여 사용하는 하이브리드 시스템:

```
use std::collections::HashMap;
use tokio::sync::RwLock;

pub struct HybridDiscovery {
services: Vec<Box<dyn Discovery>>,
cache: Arc<RwLock<HashMap<NodeId, (NodeAddr, std::time::Instant)>>>,
cache_ttl: std::time::Duration,
}
```

```
impl HybridDiscovery {
pub fn new(cache_ttl: std::time::Duration) -> Self {
Self {
services: Vec::new(),
cache: Arc::new(RwLock::new(HashMap::new())),
cache_ttl,
}
}
pub fn add_service(mut self, service: Box<dyn Discovery>) -> Self {
self.services.push(service);
self
}
/// 캐시에서 노드 정보 조회
async fn get_cached(&self, node_id: NodeId) -> Option<NodeAddr> {
let cache = self.cache.read().await;
if let Some((node_addr, timestamp)) = cache.get(&node_id) {
if timestamp.elapsed() < self.cache_ttl {
return Some(node_addr.clone());
}
}
None
}
/// 캐시에 노드 정보 저장
```

```rust
async fn cache_node_addr(&self, node_addr: &NodeAddr) {
let mut cache = self.cache.write().await;
cache.insert(node_addr.node_id, (node_addr.clone(), std::time::Instant::now()));
}
}

#[async_trait::async_trait]
impl Discovery for HybridDiscovery {
async fn publish(&self, node_addr: &NodeAddr) -> Result<(), DiscoveryError> {
// 모든 서비스에 병렬로 게시
let futures: Vec<_> = self.services.iter()
.map(|service| service.publish(node_addr))
.collect();
let results = futures::future::join_all(futures).await;
// 하나라도 성공하면 OK
let success_count = results.iter().filter(|r| r.is_ok()).count();
if success_count > 0 {
self.cache_node_addr(node_addr).await;
Ok(())
} else {
Err(DiscoveryError::Other(
anyhow::anyhow!("모든 Discovery 서비스 게시 실패").into()
))
```

```
}
}
async fn resolve(&self, node_id: NodeId) -> Option<BoxStream<Result<DiscoveryI
// 캐시 확인
if let Some(cached_addr) = self.get_cached(node_id).await {
let stream = async_stream::stream! {
yield Ok(DiscoveryItem::NodeAddr(cached_addr));
};
return Some(Box::pin(stream));
}
// 모든 서비스에서 병렬 조회
let mut streams = Vec::new();
for service in &self.services {
if let Some(stream) = service.resolve(node_id).await {
streams.push(stream);
}
}
if streams.is_empty() {
return None;
}
let cache = self.cache.clone();
let stream = async_stream::stream! {
use futures_util::stream::StreamExt;
```

```
let mut combined = futures_util::stream::select_all(streams);
while let Some(item) = combined.next().await {
match &item {
Ok(DiscoveryItem::NodeAddr(node_addr)) => {
// 캐시에 저장
let mut cache_guard = cache.write().await;
cache_guard.insert(node_addr.node_id, (node_addr.clone(), std::time::Instant::no
}
_ => {}
}
yield item;
}
};
Some(Box::pin(stream))
}
async fn subscribe(&self) -> Option<BoxStream<DiscoveryItem>> {
let mut streams = Vec::new();
for service in &self.services {
if let Some(stream) = service.subscribe().await {
streams.push(stream);
}
}
if streams.is_empty() {
```

```
return None;
}
let cache = self.cache.clone();
let stream = async_stream::stream! {
use futures_util::stream::StreamExt;
let mut combined = futures_util::stream::select_all(streams);
while let Some(item) = combined.next().await {
match &item {
DiscoveryItem::NodeAddr(node_addr) => {
// 캐시 업데이트
let mut cache_guard = cache.write().await;
cache_guard.insert(node_addr.node_id, (node_addr.clone(), std::time::Instant::no
}
_ => {}
}
yield item;
}
};
Some(Box::pin(stream))
}
}
```

2.2 새로운 프로토콜 개발 및 통합 가이드

프로토콜 핸들러 구현

복잡한 상태를 가진 프로토콜 구현 예제:

```rust
use iroh::endpoint::{Connection, Connecting};
use iroh::router::{ProtocolHandler, AcceptError};
use std::sync::Arc;
use tokio::sync::{RwLock, mpsc};
use std::collections::HashMap;

/// 분산 키-값 저장소 프로토콜
pub struct DistributedKVProtocol {
storage: Arc<RwLock<HashMap<String, Vec<u8>>>>,
peers: Arc<RwLock<HashMap<NodeId, PeerConnection>>>,
replication_factor: usize,
consistency_level: ConsistencyLevel,
}

#[derive(Debug, Clone)]
pub enum ConsistencyLevel {
Eventual, // 최종 일관성
Strong, // 강한 일관성
Quorum, // 쿼럼 기반
```

```
}

#[derive(Debug, Serialize, Deserialize)]
pub enum KVMessage {
Get { key: String },
Put { key: String, value: Vec<u8> },
Delete { key: String },
Replicate { key: String, value: Vec<u8>, version: u64 },
Response { result: KVResult },
Heartbeat { timestamp: u64 },
}

#[derive(Debug, Serialize, Deserialize)]
pub enum KVResult {
Value(Option<Vec<u8>>),
Success,
Error(String),
}

impl DistributedKVProtocol {
pub fn new(replication_factor: usize, consistency_level: ConsistencyLevel) ->
Self {
storage: Arc::new(RwLock::new(HashMap::new())),
```

```
peers: Arc::new(RwLock::new(HashMap::new())),
replication_factor,
consistency_level,
}
}
}

#[async_trait::async_trait]
impl ProtocolHandler for DistributedKVProtocol {
async fn accept(&self, connection: Connection) -> Result<(), AcceptError> {
let remote_node_id = connection.remote_node_id()
.map_err(|e| AcceptError::Other(e.into()))?;
tracing::info!("새 KV 프로토콜 연결: {}", remote_node_id);
// 피어 연결 등록
{
let mut peers = self.peers.write().await;
peers.insert(remote_node_id, PeerConnection::new(connection.clone()));
}
// 연결 처리 루프
loop {
match connection.accept_bi().await {
Ok((send, recv)) => {
let protocol = self.clone();
```

```
let remote_id = remote_node_id;
tokio::spawn(async move {
if let Err(e) = protocol.handle_stream(remote_id, send, recv).await {
tracing::error!("스트림 처리 오류: {}", e);
}
});
}
Err(e) => {
tracing::error!("스트림 수락 오류: {}", e);
break;
}
}
}
// 피어 연결 제거
{
let mut peers = self.peers.write().await;
peers.remove(&remote_node_id);
}
Ok(())
}
}

impl DistributedKVProtocol {
```

```
async fn handle_stream(
&self,
peer_id: NodeId,
mut send: quinn::SendStream,
mut recv: quinn::RecvStream,
) -> Result<(), Box<dyn std::error::Error + Send + Sync>> {
// 메시지 수신 및 처리
let message = KVMessage::recv(&mut recv).await?;
let response = match message {
KVMessage::Get { key } => {
self.handle_get(&key).await
}
KVMessage::Put { key, value } => {
self.handle_put(key, value, peer_id).await
}
KVMessage::Delete { key } => {
self.handle_delete(&key, peer_id).await
}
KVMessage::Replicate { key, value, version } => {
self.handle_replicate(key, value, version).await
}
KVMessage::Heartbeat { timestamp } => {
self.handle_heartbeat(peer_id, timestamp).await
```

```
}
_ => KVResult::Error("Unexpected message type".to_string()),
};
// 응답 전송
let response_msg = KVMessage::Response { result: response };
response_msg.send(&mut send).await?;
send.finish()?;
Ok(())
}
async fn handle_get(&self, key: &str) -> KVResult {
let storage = self.storage.read().await;
let value = storage.get(key).cloned();
KVResult::Value(value)
}
async fn handle_put(&self, key: String, value: Vec<u8>, from_peer: NodeId) ->
// 로컬 저장
{
let mut storage = self.storage.write().await;
storage.insert(key.clone(), value.clone());
}
// 복제 수행 (일관성 수준에 따라)
match self.consistency_level {
ConsistencyLevel::Eventual => {
```

```
// 비동기 복제
self.replicate_async(key, value, from_peer).await;
KVResult::Success
}
ConsistencyLevel::Strong => {
// 동기 복제
match  self.replicate_sync(key, value, from_peer).await {
Ok(_) => KVResult::Success,
Err(e) => KVResult::Error(e.to_string()),
}
}
ConsistencyLevel::Quorum => {
// 쿼럼 복제
match  self.replicate_quorum(key, value, from_peer).await {
Ok(_) => KVResult::Success,
Err(e) => KVResult::Error(e.to_string()),
}
}
}
}
}
async  fn  replicate_async(&self, key: String, value: Vec<u8>, exclude_peer: Nod
let  peers = self.peers.read().await;
let  version = std::time::SystemTime::now()
```

```
.duration_since(std::time::UNIX_EPOCH)
.unwrap()
.as_secs();
for (peer_id, peer_conn) in  peers.iter() {
if *peer_id != exclude_peer {
let  key = key.clone();
let  value = value.clone();
let  peer_conn = peer_conn.clone();
tokio::spawn(async  move {
let  msg = KVMessage::Replicate { key, value, version };
if  let Err(e) = peer_conn.send_message(msg).await {
tracing::error!("복제 실패: {}", e);
}
});
}
}
}
async  fn  replicate_sync(
&self,
key: String,
value: Vec<u8>,
exclude_peer: NodeId,
) -> Result<(), Box<dyn  std::error::Error + Send + Sync>> {
```

```
let peers = self.peers.read().await;
let version = std::time::SystemTime::now()
.duration_since(std::time::UNIX_EPOCH)
.unwrap()
.as_secs();
let mut tasks = Vec::new();
for (peer_id, peer_conn) in peers.iter() {
if *peer_id != exclude_peer {
let key = key.clone();
let value = value.clone();
let peer_conn = peer_conn.clone();
let task = tokio::spawn(async move {
let msg = KVMessage::Replicate { key, value, version };
peer_conn.send_message(msg).await
});
tasks.push(task);
}
}
// 모든 복제 완료 대기
for task in tasks {
task.await??;
}
Ok(())
```

```
}
async fn replicate_quorum(
&self,
key: String,
value: Vec<u8>,
exclude_peer: NodeId,
) -> Result<(), Box<dyn std::error::Error + Send + Sync>> {
let peers = self.peers.read().await;
let required_replicas = (self.replication_factor + 1) / 2; // 과반수
if peers.len() < required_replicas {
return Err("쿼럼을 위한 충분한 피어가 없음".into());
}
let version = std::time::SystemTime::now()
.duration_since(std::time::UNIX_EPOCH)
.unwrap()
.as_secs();
let mut tasks = Vec::new();
for (peer_id, peer_conn) in peers.iter() {
if *peer_id != exclude_peer {
let key = key.clone();
let value = value.clone();
let peer_conn = peer_conn.clone();
let task = tokio::spawn(async move {
```

```
let msg = KVMessage::Replicate { key, value, version };
peer_conn.send_message(msg).await
});
tasks.push(task);
}
}
// 쿼럼만큼 성공하면 OK
let mut success_count = 0;
for task in tasks {
if task.await.is_ok() {
success_count += 1;
if success_count >= required_replicas {
return Ok(());
}
}
}
Err("쿼럼 복제 실패".into())
}
async fn handle_delete(&self, key: &str, from_peer: NodeId) -> KVResult {
{
let mut storage = self.storage.write().await;
storage.remove(key);
}
```

```
// 삭제도 복제 필요
self.replicate_delete(key.to_string(), from_peer).await;
KVResult::Success
}
async fn replicate_delete(&self, key: String, exclude_peer: NodeId) {
// 삭제 복제 로직 (간단화)
tracing::info!("키 삭제 복제: {}", key);
}
async fn handle_replicate(&self, key: String, value: Vec<u8>, _version: u64) -
// 복제 데이터 저장 (버전 충돌 해결 로직 필요)
let mut storage = self.storage.write().await;
storage.insert(key, value);
KVResult::Success
}
async fn handle_heartbeat(&self, peer_id: NodeId, timestamp: u64) -> KVResult
tracing::debug!("하트비트 수신: {} at {}", peer_id, timestamp);
KVResult::Success
}
}


#[derive(Debug, Clone)]
struct PeerConnection {
connection: Connection,
```

```
}


impl PeerConnection {
fn new(connection: Connection) -> Self {
Self { connection }
}
async fn send_message(&self, message: KVMessage) -> Result<(), Box<dyn std::e
let (mut send, mut recv) = self.connection.open_bi().await?;
message.send(&mut send).await?;
send.finish()?;
// 응답 대기
let response = KVMessage::recv(&mut recv).await?;
match response {
KVMessage::Response { result: KVResult::Success } => Ok(()),
KVMessage::Response { result: KVResult::Error(e) } => Err(e.into()),
_ => Err("Unexpected response".into()),
}
}
}
```

2.3 성능 최적화 및 프로파일링 기법

메트릭스 기반 성능 모니터링

```rust
use std::sync::atomic::{AtomicU64, Ordering};
use std::time::{Duration, Instant};

/// 애플리케이션 성능 메트릭스
#[derive(Debug, Default)]
pub struct AppMetrics {
// 연결 메트릭스
pub connections_established: AtomicU64,
pub connections_failed: AtomicU64,
pub active_connections: AtomicU64,
// 메시지 메트릭스
pub messages_sent: AtomicU64,
pub messages_received: AtomicU64,
pub bytes_sent: AtomicU64,
pub bytes_received: AtomicU64,
// 지연시간 메트릭스
pub total_request_time: AtomicU64,
pub request_count: AtomicU64,
// 에러 메트릭스
pub timeout_errors: AtomicU64,
pub network_errors: AtomicU64,
pub protocol_errors: AtomicU64,
```

```
}


impl AppMetrics {
pub fn record_connection_established(&self) {
self.connections_established.fetch_add(1, Ordering::Relaxed);
self.active_connections.fetch_add(1, Ordering::Relaxed);
}
pub fn record_connection_closed(&self) {
self.active_connections.fetch_sub(1, Ordering::Relaxed);
}
pub fn record_message_sent(&self, bytes: u64) {
self.messages_sent.fetch_add(1, Ordering::Relaxed);
self.bytes_sent.fetch_add(bytes, Ordering::Relaxed);
}
pub fn record_request_time(&self, duration: Duration) {
self.total_request_time.fetch_add(duration.as_millis() as u64, Ordering::Relaxed
self.request_count.fetch_add(1, Ordering::Relaxed);
}
pub fn average_request_time(&self) -> f64 {
let total = self.total_request_time.load(Ordering::Relaxed);
let count = self.request_count.load(Ordering::Relaxed);
if count > 0 {
total as f64 / count as f64
```

```
} else {
0.0
}
}
/// 메트릭스를 Prometheus 형식으로 출력
pub fn to_prometheus(&self) -> String {
format!(
r#"# HELP app_connections_total Total number of connections
# TYPE app_connections_total counter
app_connections_established {{}} {}
app_connections_failed {{}} {}


# HELP app_active_connections Current number of active connections
# TYPE app_active_connections gauge
app_active_connections {{}} {}


# HELP app_messages_total Total number of messages
# TYPE app_messages_total counter
app_messages_sent {{}} {}
app_messages_received {{}} {}


# HELP app_bytes_total Total number of bytes transferred
# TYPE app_bytes_total counter
```

```
app_bytes_sent {{}} {}

app_bytes_received {{}} {}



# HELP app_request_duration_ms Average request duration in milliseconds

# TYPE app_request_duration_ms gauge

app_request_duration_ms {{}} {}



# HELP app_errors_total Total number of errors

# TYPE app_errors_total counter

app_timeout_errors {{}} {}

app_network_errors {{}} {}

app_protocol_errors {{}} {}

"#,

self.connections_established.load(Ordering::Relaxed),

self.connections_failed.load(Ordering::Relaxed),

self.active_connections.load(Ordering::Relaxed),

self.messages_sent.load(Ordering::Relaxed),

self.messages_received.load(Ordering::Relaxed),

self.bytes_sent.load(Ordering::Relaxed),

self.bytes_received.load(Ordering::Relaxed),

self.average_request_time(),

self.timeout_errors.load(Ordering::Relaxed),

self.network_errors.load(Ordering::Relaxed),
```

```
self.protocol_errors.load(Ordering::Relaxed),
)
}
}

/// 성능 측정 래퍼
pub struct PerformanceTracker {
metrics: Arc<AppMetrics>,
}

impl PerformanceTracker {
pub fn new(metrics: Arc<AppMetrics>) -> Self {
Self { metrics }
}
/// 요청 시간 측정
pub async fn track_request<F, T>(&self, operation: F) -> T
where
F: std::future::Future<Output = T>,
{
let start = Instant::now();
let result = operation.await;
let duration = start.elapsed();
self.metrics.record_request_time(duration);
```

```
result
}
/// 연결 생명주기 추적
pub fn track_connection(&self) -> ConnectionTracker {
self.metrics.record_connection_established();
ConnectionTracker {
metrics: self.metrics.clone(),
}
}
}


pub struct ConnectionTracker {
metrics: Arc<AppMetrics>,
}


impl Drop for ConnectionTracker {
fn drop(&mut self) {
self.metrics.record_connection_closed();
}
}
```

프로파일링 및 벤치마킹

```
use criterion::{criterion_group, criterion_main, Criterion, BenchmarkId};
```

```rust
use tokio::runtime::Runtime;


/// 연결 설정 성능 벤치마크
fn benchmark_connection_setup(c: &mut Criterion) {
let rt = Runtime::new().unwrap();
c.bench_function("connection_setup", |b| {
b.to_async(&rt).iter(|| async {
let endpoint1 = Endpoint::builder().bind().await.unwrap();
let endpoint2 = Endpoint::builder().bind().await.unwrap();
let node_addr = NodeAddr::new(endpoint2.node_id())
.with_direct_addresses(endpoint2.bound_sockets());
let _connection = endpoint1.connect(node_addr, b"bench").await.unwrap();
});
});
}


/// 메시지 처리량 벤치마크
fn benchmark_message_throughput(c: &mut Criterion) {
let rt = Runtime::new().unwrap();
let mut group = c.benchmark_group("message_throughput");
for message_size in [1024, 4096, 16384, 65536].iter() {
group.bench_with_input(
BenchmarkId::new("send_receive", message_size),
```

```
message_size,
|b, &size| {
b.to_async(&rt).iter(|| async  move {
let (endpoint1, endpoint2) = setup_connected_endpoints().await;
let  message = vec![0u8; size];
let  start = Instant::now();
send_and_receive_message(&endpoint1, &endpoint2, message).await;
start.elapsed()
});
},
);
}
group.finish();
}


async  fn  setup_connected_endpoints() -> (Endpoint, Endpoint) {
let  endpoint1 = Endpoint::builder().bind().await.unwrap();
let  endpoint2 = Endpoint::builder().bind().await.unwrap();
// 연결 설정 로직
(endpoint1, endpoint2)
}


async  fn  send_and_receive_message(
```

```
sender: &Endpoint,
receiver: &Endpoint,
message: Vec<u8>,
) {
// 메시지 송수신 로직
}

criterion_group!(benches, benchmark_connection_setup, benchmark_message_throughp
criterion_main!(benches);
```

메모리 사용량 최적화

```
use std::alloc::{GlobalAlloc, Layout, System};
use std::sync::atomic::{AtomicUsize, Ordering};

/// 메모리 사용량 추적 할당자
pub struct TrackingAllocator {
allocated: AtomicUsize,
peak_allocated: AtomicUsize,
}

impl TrackingAllocator {
pub const fn new() -> Self {
```

```
Self {
allocated: AtomicUsize::new(0),
peak_allocated: AtomicUsize::new(0),
}
}
pub fn allocated(&self) -> usize {
self.allocated.load(Ordering::Relaxed)
}
pub fn peak_allocated(&self) -> usize {
self.peak_allocated.load(Ordering::Relaxed)
}
pub fn reset_peak(&self) {
self.peak_allocated.store(self.allocated.load(Ordering::Relaxed), Ordering::Rela
}
}


unsafe impl GlobalAlloc for TrackingAllocator {
unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
let ptr = System.alloc(layout);
if !ptr.is_null() {
let size = layout.size();
let old_allocated = self.allocated.fetch_add(size, Ordering::Relaxed);
let new_allocated = old_allocated + size;
```

```
// 피크 메모리 사용량 업데이트
let mut peak = self.peak_allocated.load(Ordering::Relaxed);
while peak < new_allocated {
match self.peak_allocated.compare_exchange_weak(
peak,
new_allocated,
Ordering::Relaxed,
Ordering::Relaxed,
) {
Ok(_) => break,
Err(x) => peak = x,
}
}
}
ptr
}
unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout) {
System.dealloc(ptr, layout);
self.allocated.fetch_sub(layout.size(), Ordering::Relaxed);
}
}


#[global_allocator]
```

```rust
static ALLOCATOR: TrackingAllocator = TrackingAllocator::new();


/// 메모리 사용량 모니터링
pub struct MemoryMonitor {
interval: Duration,
shutdown: CancellationToken,
}


impl MemoryMonitor {
pub fn new(interval: Duration) -> Self {
Self {
interval,
shutdown: CancellationToken::new(),
}
}
pub async fn start_monitoring(&self) {
let mut interval_timer = tokio::time::interval(self.interval);
loop {
tokio::select! {
_ = self.shutdown.cancelled() => break,
_ = interval_timer.tick() => {
let allocated = ALLOCATOR.allocated();
let peak = ALLOCATOR.peak_allocated();
```

```
tracing::info!(
allocated_mb = allocated / 1024 / 1024,
peak_mb = peak / 1024 / 1024,
"메모리 사용량"
);
// 메모리 사용량이 임계값을 초과하면 경고
if allocated > 1024 * 1024 * 1024 { // 1GB
tracing::warn!("높은 메모리 사용량 감지: {}MB", allocated / 1024 / 1024);
}
}
}
}
}
pub fn shutdown(&self) {
self.shutdown.cancel();
}
}
```

CPU 프로파일링 통합

```
#[cfg(feature = "profiling")]
use pprof::ProfilerGuard;

pub struct Profiler {
```

```
#[cfg(feature = "profiling")]
guard: Option<ProfilerGuard<'static>>,
}


impl Profiler {
pub fn new() -> Self {
Self {
#[cfg(feature = "profiling")]
guard: None,
}
}
pub fn start(&mut self) -> Result<(), Box<dyn std::error::Error>> {
#[cfg(feature = "profiling")]
{
let guard = pprof::ProfilerGuardBuilder::default()
.frequency(1000) // 1000Hz 샘플링
.blocklist(&["libc", "libgcc", "pthread", "vdso"])
.build()?;
self.guard = Some(guard);
}
Ok(())
}
pub fn stop_and_report(&mut self) -> Result<(), Box<dyn std::error::Error>>
```

```
#[cfg(feature = "profiling")]
{
if let Some(guard) = self.guard.take() {
match guard.report().build() {
Ok(report) => {
let file = std::fs::File::create("profile.pb")?;
let mut content = Vec::new();
report.pprof()?.write_to_vec(&mut content)?;
std::io::Write::write_all(&mut std::io::BufWriter::new(file), &content)?;
tracing::info!("프로파일 저장됨: profile.pb");
}
Err(e) => tracing::error!("프로파일 생성 실패: {}", e),
}
}
}
Ok(())
}
}

/// 프로파일링을 포함한 애플리케이션 래퍼
pub struct ProfiledApplication {
app: Application,
profiler: Profiler,
```

```
memory_monitor: MemoryMonitor,
}

impl ProfiledApplication {
pub fn new(app: Application) -> Self {
Self {
app,
profiler: Profiler::new(),
memory_monitor: MemoryMonitor::new(Duration::from_secs(10)),
}
}
pub async fn run_with_profiling(&mut self) -> Result<(), Box<dyn std::error
// 프로파일링 시작
self.profiler.start()?;
// 메모리 모니터링 시작
let memory_monitor = self.memory_monitor.clone();
tokio::spawn(async move {
memory_monitor.start_monitoring().await;
});
// 애플리케이션 실행
let result = self.app.run().await;
// 프로파일링 종료 및 보고서 생성
self.profiler.stop_and_report()?;
```

```
self.memory_monitor.shutdown();

result

}

}
```

이러한 고급 기법들을 활용하면 iroh 기반 애플리케이션의 성능을 최적화하고, 확장 가능한 아키텍처를 구축할 수 있습니다.## 2. 고급 활

용 및 확장 가이드

2.1 커스텀 Discovery 구현 방법

iroh의 Discovery 시스템은 확장 가능한 아키텍처를 제공하여 커스텀 노드 발견 메커니즘을 구현할 수 있습니다.

Discovery 트레이트 구현

```
use iroh::discovery::{Discovery, DiscoveryItem, DiscoveryError};

use iroh_base::{NodeId, NodeAddr};

use futures_lite::Stream;

use std::pin::Pin;


/// 커스텀 Discovery 서비스 예제 - 중앙 집중식 레지스트리

pub  struct CentralizedDiscovery {

registry_url: String,

http_client: reqwest::Client,

node_id: NodeId,
```

```
}

impl CentralizedDiscovery {
pub fn new(registry_url: String, node_id: NodeId) -> Self {
Self {
registry_url,
http_client: reqwest::Client::new(),
node_id,
}
}
}

#[async_trait::async_trait]
impl Discovery for CentralizedDiscovery {
/// 노드 정보를 중앙 레지스트리에 게시
async fn publish(&self, node_addr: &NodeAddr) -> Result<(), DiscoveryError> {
let url = format!("{}/nodes/{}", self.registry_url, node_addr.node_id);
let payload = serde_json::json!({
"node_id": node_addr.node_id.to_string(),
"relay_url": node_addr.relay_url().map(|u| u.to_string()),
"direct_addresses": node_addr.direct_addresses().collect::<Vec<_>>(),
"timestamp": std::time::SystemTime::now()
.duration_since(std::time::UNIX_EPOCH)
```

```
.unwrap()
.as_secs()
});
let response = self.http_client
.put(&url)
.json(&payload)
.send()
.await
.map_err(|e| DiscoveryError::Other(e.into()))?;
if !response.status().is_success() {
return Err(DiscoveryError::Other(
anyhow::anyhow!("Failed to publish: {}", response.status()).into()
));
}
tracing::info!("노드 정보 게시 완료: {}", node_addr.node_id);
Ok(())
}
/// 중앙 레지스트리에서 노드 정보 조회
async fn resolve(&self, node_id: NodeId) -> Option<BoxStream<Result<DiscoveryI
let url = format!("{}/nodes/{}", self.registry_url, node_id);
let stream = async_stream::stream! {
match self.http_client.get(&url).send().await {
Ok(response) if response.status().is_success() => {
```

```
match response.json::<serde_json::Value>().await {
Ok(data) => {
if let Some(node_addr) = self.parse_node_addr(&data) {
yield Ok(DiscoveryItem::NodeAddr(node_addr));
}
}
Err(e) => yield Err(DiscoveryError::Other(e.into())),
}
}
Ok(response) => {
yield Err(DiscoveryError::Other(
anyhow::anyhow!("HTTP error: {}", response.status()).into()
));
}
Err(e) => yield Err(DiscoveryError::Other(e.into())),
}
};
Some(Box::pin(stream))
}
/// 노드 변경 사항 구독 (폴링 방식)
async fn subscribe(&self) -> Option<BoxStream<DiscoveryItem>> {
let registry_url = self.registry_url.clone();
let http_client = self.http_client.clone();
```

```
let stream = async_stream::stream! {
let mut last_check = std::time::SystemTime::UNIX_EPOCH;
let mut interval = tokio::time::interval(std::time::Duration::from_secs(30));
loop {
interval.tick().await;
let url = format!("{}/nodes/changes?since={}",
registry_url,
last_check.duration_since(std::time::UNIX_EPOCH).unwrap().as_secs()
);
match http_client.get(&url).send().await {
Ok(response) if response.status().is_success() => {
match response.json::<Vec<serde_json::Value>>().await {
Ok(changes) => {
for change in changes {
if let Some(node_addr) = self.parse_node_addr(&change) {
yield DiscoveryItem::NodeAddr(node_addr);
}
}
last_check = std::time::SystemTime::now();
}
Err(e) => tracing::error!("구독 파싱 오류: {}", e),
}
}
```

```
Ok(response) => {
tracing::error!("구독 HTTP 오류: {}", response.status());
}
Err(e) => tracing::error!("구독 네트워크 오류: {}", e),
}
}
};
Some(Box::pin(stream))
}
}


impl CentralizedDiscovery {
fn  parse_node_addr(&self, data: &serde_json::Value) -> Option<NodeAddr> {
let  node_id_str = data.get("node_id")?.as_str()?;
let  node_id = NodeId::from_str(node_id_str).ok()?;
let  mut  node_addr = NodeAddr::new(node_id);
// Relay URL 추가
if  let Some(relay_url_str) = data.get("relay_url").and_then(|v| v.as_str()) {
if  let Ok(relay_url) = relay_url_str.parse() {
node_addr = node_addr.with_relay_url(relay_url);
}
}
// 직접 주소들 추가
```

```
if let Some(direct_addrs) = data.get("direct_addresses").and_then(|v| v.as_arra

for addr_value in direct_addrs {

if let Some(addr_str) = addr_value.as_str() {

if let Ok(socket_addr) = addr_str.parse() {

node_addr = node_addr.with_direct_addresses([socket_addr]);

}

}

}

}

Some(node_addr)

}

}
```

이 개발자 가이드는 iroh를 활용한 P2P 애플리케이션 개발의 모든 측면을 다루며, 기본적인 연결 설정부터 고급 최적화 기법, 그리고 실제 사용 사례까지 포괄적으로 설명합니다. 각 예제는 실제 프로덕션 환경에서 사용할 수 있도록 에러 처리, 성능 최적화, 확장성을 고려하여 구현되었습니다.